

数据结构与算法 实验报告

第四次



姓名 陈宇昂

班级 数学强基 2202 班

学号 2224110096

电话 **

Email yuang.chen2024@outlook.com

日期 2024-11-30

目录

任务 1: 构建逻辑示意图	3
一、 题目	3
二、 数据结构设计	3
(一) WordLadderGraph 类	3
(二) 顶点与边	3
三、 逻辑示意图	3
任务 2: Graph 类的设计与实现	4
一、 题目	4
二、 算法设计	4
(一) 添加顶点 addWord()	4
(二) 添加边 addEdge()	4
(三) 判断单词差异 isOneLetterDifference()	4
(四) 构建图 buildGraph()	4
(五) 路径检查 isPathExist()	4
三、 主干代码说明	5
(一) WordLadderGraph 类	5
任务 3: 交互式字梯游戏	7
子任务一:	7
一、 题目: 找出无法形成字梯的单词	7
二、 算法设计	7
(一) 读取单词列表	7
(二) 构建单词图	7
(三) 生成无法形成字梯的单词列表	7
(四) 输出无连接单词	7
三、 主干代码说明	7
四、 运行结果展示	8
子任务二:	9
一、 题目: 交互式字梯游戏	9
二、 算法设计	9
(一) 读取单词列表	9
(二) 构建单词图	9
(三) 随机选择单词对	9
(四) 用户交互	9
(五) “我不会” 功能	9
(六) 游戏终止条件	9
三、 主干代码说明	10
四、 运行结果展示	11
五、 总结与收获	11

任务 1：构建逻辑示意图

一、题目

Word Ladder（字梯，又称 Doublets）是由 Lewis Carroll 发明的一种文字游戏。一个单词梯形谜题由两个单词开始，要解开谜题，必须找到一个其他单词链来连接这两个单词，其中相邻的两个单词（即连续步骤中的单词）相差一个字母。

Lewis Carroll 说，他在 1877 年的圣诞节发明了这个游戏。Carroll 为 Julia 和 Ethel Arnold 设计了这个文字游戏，Carroll 的日记中第一次提到这个游戏是在 1878 年 3 月 12 日，他最初把这个游戏叫做“单词-连线”，并把它描述为一个双人游戏。Carroll 从 1879 年 3 月 29 日的《名利场》杂志开始，发表了一系列的字梯谜题和解法，他当时称之为“Doublets”。

这个游戏的规则是：玩家会得到一个起始词和一个结束词。为了赢得游戏，玩家必须将开始的单词逐步改变成结束的单词，每一步都要选择一个现有的单词，而被选的这个词与上一步的词之间只有一个字母的差异。例如，下面是用柯林斯拼字游戏中的单词来解决“COLD”和“WARM”之间的单词字梯谜题的七个最短解法：

```
COLD → CORD → CARD → WARD → WARM
COLD → CORD → CORM → WORM → WARM
COLD → CORD → WORD → WARD → WARM
COLD → CORD → WORD → WORM → WARM
COLD → WOLD → WALD → WARD → WARM
COLD → WOLD → WORD → WARD → WARM
COLD → WOLD → WORD → WORM → WARM
```

建立为实现该游戏的图的抽象描述结构，包括图中顶点的意义以及存储的信息、边的意义以及存储的信息，并给出该图的逻辑示意图。

二、数据结构设计

（一）WordLadderGraph 类

Map<String, List<String>> adjacencyList: 使用哈希映射表示图的邻接表结构。键为 String 类型的单词，值为与该单词相邻的单词列表。

addWord(String word): 将一个单词作为顶点加入图中。

addEdge(String word1, String word2): 如果两个单词仅相差一个字母，则在它们之间添加一条无向边。

isOneLetterDifference(String word1, String word2): 判断两个单词是否相差一个字母，用于判断是否添加边。

buildGraph(List<String> words): 根据输入的单词列表构建完整的图结构。

（二）顶点与边

顶点：每个顶点代表一个单词，存储该单词的字符串信息。

边：每条边表示其两端顶点之间的单词相差一个字母，存储邻接关系，不需要额外信息。

三、逻辑示意图

例如：COLD → CORD → CARD → WARD

使用邻接表存储，即为：

```
COLD → [CORD]
CORD → [COLD, CARD]
CARD → [CORD, WARD]
WARD → [CARD]
```

任务 2: Graph 类的设计与实现

一、题目

在任务 1 的基础上，结合教材中图的抽象数据类型的定义，设计并实现一个为该游戏而使用的具体的 Graph Class。

二、算法设计

（一）添加顶点 addWord()

addWord(String word):

- ① 使用 `HashMap<String, List<String>>` 存储单词图的邻接表，单词作为键，相邻单词列表作为值。
- ② 调用 `putIfAbsent()` 方法，确保新单词仅在未存在时加入图中。
- ③ 该方法用于初始化图中的顶点。

（二）添加边 addEdge()

addEdge(String word1, String word2):

- ① 判断 `word1` 与 `word2` 是否相差一个字母，调用 `isOneLetterDifference()`。
- ② 若条件成立，使用邻接表将 `word1` 和 `word2` 互相添加为邻居，实现无向边的存储。
- ③ 通过双向更新，实现两单词之间的连接。

（三）判断单词差异 isOneLetterDifference()

isOneLetterDifference(String word1, String word2):

- ① 比较两单词长度，若不相等，返回 `false`。
- ② 遍历每个字母，统计差异计数器 `differences`。
- ③ 差异为 1 时返回 `true`，否则返回 `false`。

（四）构建图 buildGraph()

buildGraph(List<String> words):

- ① 调用 `addWord()` 初始化所有单词为顶点。
- ② 遍历单词对，通过 `addEdge()` 建立所有合法边。
- ③ 构建完成的图存储在 `adjacencyList` 中。

（五）路径检查 isPathExist()

isPathExist(String startWord, String endWord):

- ① 使用 BFS 查找从 `startWord` 到 `endWord` 的路径。
- ② 若路径存在，返回 `true`，否则返回 `false`，实现路径连通性验证。

三、主干代码说明

(一) WordLadderGraph 类

```

1      import java.util.*;
2
3      public class WordLadderGraph { 5个用法
4
5          private Map<String, List<String>> adjacencyList; 6个用法
6
7          public WordLadderGraph() { 2个用法
8              this.adjacencyList = new HashMap<>();
9          }
    
```

1、addWord(String word):

```

11     public void addWord(String word) { 1个用法
12         adjacencyList.putIfAbsent(word, new ArrayList<>());
13     }
    
```

2、addEdge(String word1, String word2):

```

15     public void addEdge(String word1, String word2) {
16         if (isOneLetterDifference(word1, word2)) {
17             adjacencyList.get(word1).add(word2);
18             adjacencyList.get(word2).add(word1);
19         }
20     }
    
```

3、isOneLetterDifference(String word1, String word2):

```

22     public boolean isOneLetterDifference(String word1, String word2) {
23         if (word1.length() != word2.length()) return false;
24         int differences = 0;
25         for (int i = 0; i < word1.length(); i++) {
26             if (word1.charAt(i) != word2.charAt(i)) {
27                 differences++;
28                 if (differences > 1) return false;
29             }
30         }
31         return differences == 1;
32     }
    
```

4、buildGraph(List<String> words):

```

34 @      public void buildGraph(List<String> words) {
35         for (String word : words) {
36             addWord(word);
37         }
38         for (String word1 : words) {
39             for (String word2 : words) {
40                 if (!word1.equals(word2)) {
41                     addEdge(word1, word2);
42                 }
43             }
44         }
45     }

```

5、getAdjacencyList():

```

47     public Map<String, List<String>> getAdjacencyList() { 4个用法
48         return adjacencyList;
49     }

```

6、isPathExist(String startWord, String endWord):

```

51     public boolean isPathExist(String startWord, String endWord) {
52         Set<String> visited = new HashSet<>();
53         Queue<String> queue = new LinkedList<>();
54         queue.offer(startWord);
55         while (!queue.isEmpty()) {
56             String current = queue.poll();
57             if (current.equals(endWord)) return true;
58             visited.add(current);
59             for (String neighbor : adjacencyList.get(current)) {
60                 if (!visited.contains(neighbor)) {
61                     queue.offer(neighbor);
62                 }
63             }
64         }
65         return false;
66     }
67 }

```


任务 3：交互式字梯游戏

该任务中会提供一个所有长度为 5 的单词列表文件 words5.txt，需要针对提供的这个单词列表解决如下问题：

子任务一：

一、题目：找出无法形成字梯的单词

- ① 针对 words5.txt 文件中的单词列表，生成一个 noladder.txt 文件，该文件中记录的单词是无法和其他单词形成字梯的所有单词。

二、算法设计

（一）读取单词列表

文件读取：

- ① 使用 BufferedReader 读取 words5.txt 文件，每行读取一个单词。
- ② 将每个单词去除空格后加入 List<String> 中，供后续图的构建使用。
- ③ 捕获 IOException 异常，保证文件读取过程的稳定性。

（二）构建单词图

图构建逻辑：

- ① 使用 WordLadderGraph 类，通过 buildGraph() 方法，将单词列表构建为邻接表。
- ② 每个单词作为顶点，相差一个字母的单词之间建立无向边。

（三）生成无法形成字梯的单词列表

遍历与检查：

- ① 遍历 adjacencyList 中的每个单词及其邻接列表。
- ② 若某单词的邻接列表为空，则该单词无法与其他单词形成字梯。

（四）输出无连接单词

文件写入：

- ① 使用 BufferedWriter 创建 noladder.txt 文件，将无法连接的单词逐行写入。
- ② 每个单词写入后换行，最后输出文件生成成功的提示信息。
- ③ 捕获 IOException 异常，确保写入过程无误。

三、主干代码说明

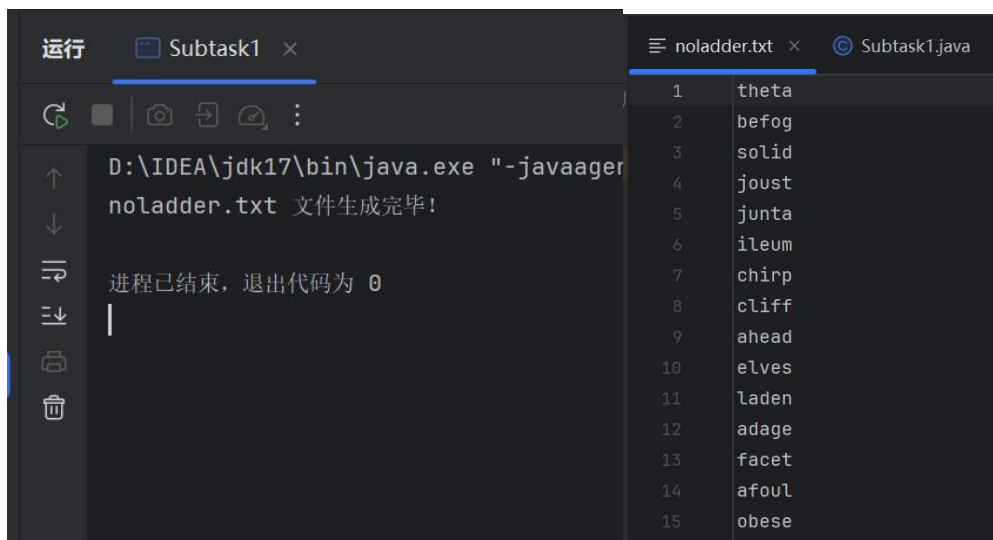
```
1  import java.io.*;
2  import java.util.*;
3
4  public class Subtask1 {
5      public static void main(String[] args) {
6          WordLadderGraph graph = new WordLadderGraph();
7          List<String> words = new ArrayList<>();
```

```

9      // 读取 words5.txt 文件
10     try (BufferedReader br = new BufferedReader(new FileReader( fileName: "words5.txt"))) {
11         String line;
12         while ((line = br.readLine()) != null) {
13             words.add(line.trim());
14         }
15     } catch (IOException e) {
16         e.printStackTrace();
17     }
18
19     // 构建图
20     graph.buildGraph(words);
21
22     // 生成无法连接的单词列表
23     try (BufferedWriter writer = new BufferedWriter(new FileWriter( fileName: "noladder.txt"))) {
24         for (String word : graph.getAdjacencyList().keySet()) {
25             if (graph.getAdjacencyList().get(word).isEmpty()) {
26                 writer.write(word);
27                 writer.newLine();
28             }
29         }
30         System.out.println("noladder.txt 文件生成完毕!");
31     } catch (IOException e) {
32         e.printStackTrace();
33     }
34 }
35 }

```

四、运行结果展示



说明 Subtask1 的实现完全正确

子任务二：

一、题目：交互式字梯游戏

- ② 编写一个具有交互功能的程序，给用户随机抽两个单词（注：这两个单词必须要保证能够有字梯链），接受用户的输入，判断用户的每次输入是否是正确的，直到用户失败或者成功。（如果可能，还可以增加判断用户的成功输入是否是最短的变化链路）

二、算法设计

（一）读取单词列表

文件读取：

- ① 使用 Scanner 读取 words5.txt 文件，每行读取一个单词。
- ② 将单词去除空格后加入 List<String> 中供后续处理。
- ③ 处理 FileNotFoundException 异常，确保文件正常读取。

（二）构建单词图

图构建：

- ① 使用 WordLadderGraph 类，通过 buildGraph() 方法，构建图的邻接表。
- ② 每个单词为顶点，相差一个字母的单词通过边相连。

（三）随机选择单词对

选择逻辑：

- ① 使用 Random 随机选择两个单词。
- ② 调用 isPathExist() 方法，确保两单词之间存在路径。
- ③ 如果路径不存在，继续随机选择，直到找到可连接的单词对。

（四）用户交互

输入验证与反馈：

- ① 提示用户从起始单词逐步输入相邻单词，直到完成字梯或输入错误。
- ② 检查用户输入是否在 words5.txt 中，若不在，则提示“无效输入，您输入的单词不在 words5 中”。
- ③ 判断用户输入的单词是否与前一单词只差一个字母，若错误，提示“无效输入，请确保单词仅差一个字母”。

（五）“我不会”功能

路径显示：

- ① 用户输入“我不会”时，程序调用 findShortestPath() 使用 BFS 查找并显示起始单词到目标单词的最短路径。
- ② BFS 通过记录前驱节点生成路径，保证最短路径输出。

（六）游戏终止条件

成功与失败判定：

- ① 当用户输入的单词与目标单词匹配时，提示“恭喜你成功完成字梯！”并结束游戏。
- ② 若用户选择退出或输入错误导致游戏无法继续，游戏结束。

三、主干代码说明

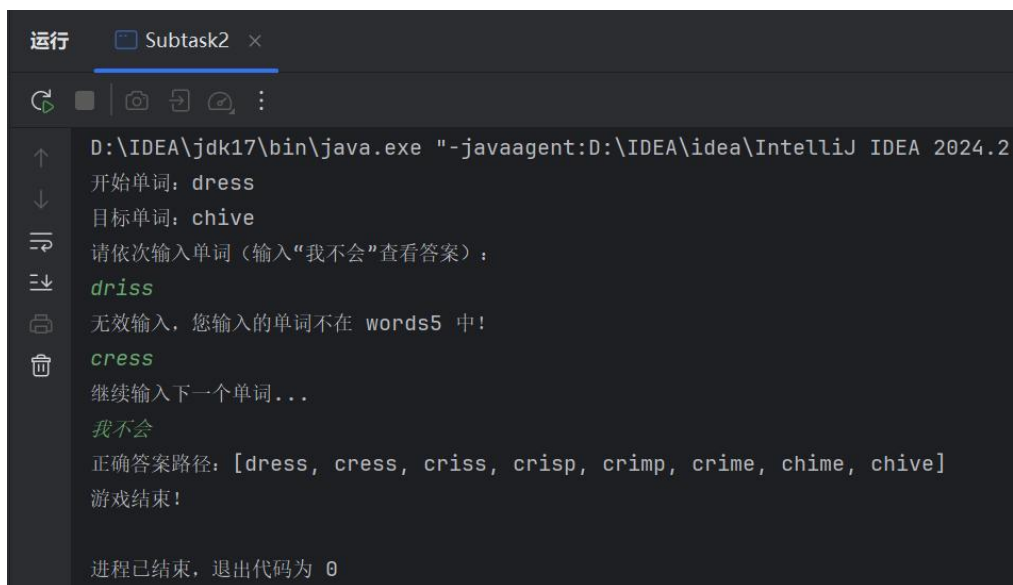
```

1  import java.util.*;
2  import java.io.*;
3
4  public class Subtask2 {
5      public static void main(String[] args) {
6          WordLadderGraph graph = new WordLadderGraph();
7          List<String> words = new ArrayList<>();
8
9          // 读取 words5.txt 文件
10         try (Scanner scanner = new Scanner(new File( pathname: "words5.txt"))) {
11             while (scanner.hasNextLine()) {
12                 words.add(scanner.nextLine().trim());
13             }
14         } catch (FileNotFoundException e) {
15             System.err.println("文件 words5.txt 未找到! ");
16             return;
17         }
18
19         // 构建图
20         graph.buildGraph(words);
21
22         // 选择两个可连接的单词
23         List<String> wordList = new ArrayList<>(graph.getAdjacencyList().keySet());
24         Random random = new Random();
25         String startWord, endWord;
26         do {
27             startWord = wordList.get(random.nextInt(wordList.size()));
28             endWord = wordList.get(random.nextInt(wordList.size()));
29         } while (!graph.isPathExist(startWord, endWord));
30
31         // 交互式游戏
32         Scanner inputScanner = new Scanner(System.in);
33         System.out.println("开始单词: " + startWord);
34         System.out.println("目标单词: " + endWord);
35         System.out.println("请依次输入单词 (输入“我不会”查看答案): ");
36
37         String currentWord = startWord;
38         boolean success = false;
39         while (true) {
40             String userInput = inputScanner.nextLine().trim();
41
42             // 检查用户是否请求显示答案
43             if (userInput.equals("我不会")) {
44                 List<String> solutionPath = findShortestPath(graph, startWord, endWord);
45                 System.out.println("正确答案路径: " + solutionPath);
46                 break;
47             }
48
49             // 检查输入是否合法
50             if (!words.contains(userInput)) {
51                 System.out.println("无效输入, 您输入的单词不在 words5 中!");
52             } else if (graph.isOneLetterDifference(currentWord, userInput)) {
53                 currentWord = userInput;
54                 if (currentWord.equals(endWord)) {
55                     System.out.println("恭喜你成功完成字梯!");
56                     success = true;
57                     break;
58                 } else {
59                     System.out.println("继续输入下一个单词...");
60                 }
61             } else {
62                 System.out.println("无效输入, 请确保单词仅差一个字母!");
63             }
64         }
65         if (!success) {
66             System.out.println("游戏结束!");
67         }
68         inputScanner.close();
69     }

```

```
71 // 使用广度优先搜索 (BFS) 查找最短路径
72 @ private static List<String> findShortestPath(WordLadderGraph graph, String startWord, String endWord) { 1个用法
73     Map<String, String> previous = new HashMap<>();
74     Queue<String> queue = new LinkedList<>();
75     Set<String> visited = new HashSet<>();
76
77     queue.offer(startWord);
78     visited.add(startWord);
79
80     while (!queue.isEmpty()) {
81         String current = queue.poll();
82         if (current.equals(endWord)) {
83             break;
84         }
85         for (String neighbor : graph.getAdjacencyList().get(current)) {
86             if (!visited.contains(neighbor)) {
87                 visited.add(neighbor);
88                 previous.put(neighbor, current);
89                 queue.offer(neighbor);
90             }
91         }
92     }
93
94     // 生成路径
95     List<String> path = new LinkedList<>();
96     for (String at = endWord; at != null; at = previous.get(at)) {
97         path.add(index 0, at);
98     }
99     return path;
100 }
101 }
```

四、运行结果展示



```
运行 Subtask2 x
D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2
开始单词: dress
目标单词: chive
请依次输入单词 (输入“我不会”查看答案):
driss
无效输入, 您输入的单词不在 words5 中!
cress
继续输入下一个单词...
我不会
正确答案路径: [dress, cress, criss, crisp, crimp, crime, chime, chive]
游戏结束!
进程已结束, 退出代码为 0
```

说明交互字梯游戏的实现完全正确

五、总结与收获

在本次实验中,通过图的构建与路径搜索算法的实现,我体会到了图数据结构在复杂问题求解中的高效性。使用邻接表存储单词图,既节省空间又便于操作,实现了单词间路径的快速查询。

在生成 noladder.txt 时,遍历孤立顶点筛选出无法形成字梯的单词,展现了图算法的灵活性。字梯游戏中,通过 BFS 确保单词对之间路径的连通性,并实时验证用户输入的合法性,提高了交互体验。“我不会”功能借助 BFS 输出最短路径,进一步体现了图在路径优化中的优势。本次实验深化了我对图结构及其在路径搜索、单词链生成等实际应用中的理解,同时提升了我算法设计和面向对象编程能力。